

CHAPTER 1: PYTHON BASICS

1. Entering expressions into the interactive shell
2. The integer, floating-point and String Data Types
3. String concatenation and replication
4. Storing values in variables
5. Your first program
6. Dissecting your program

1.1. Entering expressions into the interactive shell

- Run the interactive shell by launching IDLE, which is installed with Python. On Windows, open the Start menu, select **All Programs** ▶ **Python 3.3**, and then select **IDLE (Python GUI)**. On OS X, select **Applications** ▶ **MacPython 3.3** ▶ **IDLE**. On Ubuntu, open a new Terminal window and enter **idle3**.
- A window with the `>>>` prompt should appear; that's the interactive shell.

```
>>> 2+2
4
```

- The IDLE window should now show some text like this:

```
Python 3.3.2 (v3.3.2: d047928ae3f6, May 16 2013, 00:06:53) [MSC v.1600 64 bit
(AMD64)] on win32
```

- Type "copyright", "credits" or "license ()" for more information.

```
>>> 2+2
4
```

- In Python, $2 + 2$ is called an *expression*, which is the most basic kind of programming instruction in the language. Expressions consist of *values* (such as 2) and *operators* (such as +), and they can always *evaluate* (that is, reduce) down to a single value. That means you can use expressions anywhere in Python code that you could also use a value.
- In the previous example, $2 + 2$ is evaluated down to a single value, 4. A single value with no operators is also considered an expression, though it evaluates only to itself, as shown here:

```
>>> 2
2
```

- The other operators which can be used are:

Operator	Operation	Example	Evaluates to...
**	Exponent	2 ** 3	8
%	Modulus/remainder	22 % 8	6
//	Integer division/floored quotient	22 // 8	2
/	Division	22 / 8	2.75
*	Multiplication	3 * 5	15
-	Subtraction	5 - 2	3
+	Addition	2 + 2	4

- The order of operations (also called precedence) of Python math operators is similar to that of mathematics. The `**` operator is evaluated first; the `*`, `/`, `//`, and `%` operators are evaluated next, from left to right; and the `+` and `-` operators are evaluated last (also from left to right). We can use parentheses to override the usual precedence if you need to.

```
>>> 2 + 3 * 6
20
>>> (2 + 3) * 6
30
>>> 48565878 * 578453
28093077826734
>>> 2 ** 8
256
>>> 23 / 7
3.2857142857142856
>>> 23 // 7
3
>>> 23 % 7
2
>>> 2 + 2
4
>>> (5 - 1) * ((7 + 1) / (3 - 1))
16.0
```

```
(5 - 1) * ((7 + 1) / (3 - 1))
↓
4 * ((7 + 1) / (3 - 1))
↓
4 * ( 8 ) / (3 - 1))
↓
4 * ( 8 ) / ( 2 )
↓
4 * 4.0
↓
16.0
```

Figure 1-1: Evaluating an expression reduces it to a single value.

- Due to wrong instructions errors occurs as shown below:

```
>>> 5 +
File "<stdin>", line 1
  5 +
    ^
SyntaxError: invalid syntax
>>> 42 + 5 + * 2
File "<stdin>", line 1
  42 + 5 + * 2
          ^
SyntaxError: invalid syntax
```

The integer, floating-point and String Data Types

- The expressions are just values combined with operators, and they always evaluate down to a single value.
- A data type is a category for values, and every value belongs to exactly one data type.

Table 1-2: Common Data Types

Data type	Examples
Integers	-2, -1, 0, 1, 2, 3, 4, 5
Floating-point numbers	-1.25, -1.0, --0.5, 0.0, 0.5, 1.0, 1.25
Strings	'a', 'aa', 'aaa', 'Hello!', '11 cats'

- The integer (or int) data type indicates values that are whole numbers.
- Numbers with a decimal point, such as 3.14, are called floating-point numbers (or floats).
- Note that even though the value 42 is an integer, the value 42.0 would be a floating-point number.
- Python programs can also have text values called strings, or strs and surrounded in single quote.
- The string with no characters, "", called a blank string.
- If the error message Syntax Error: EOL while scanning string literal, then probably the final single quote character at the end of the string is missing.

```
>>> 'Hello world!
SyntaxError: EOL while scanning string literal
```

String concatenation and replication

- The meaning of an operator may change based on the data types of the values next to it.
- For example, + is the addition operator when it operates on two integers or floating-point values.
- However, when + is used on two string values, it joins the strings as the string concatenation operator.

```
>>> 2 + 2
4
```

```
>>> 'Alice' + 'Bob'
'AliceBob'
```

- If we try to use the + operator on a string and an integer value, Python will not know how to handle this, and it will display an error message.

```
>>> 'Alice' + 42
Traceback (most recent call last):
  File "<pyshell#26>", line 1, in <module>
    'Alice' + 42
TypeError: Can't convert 'int' object to str implicitly
```

- The * operator is used for multiplication when it operates on two integer or floating-point values.
- But, when the * operator is used on one string value and one integer value, it becomes the string replication operator.

```
>>> 'Alice' * 5
'AliceAliceAliceAliceAlice'
```

- The * operator can be used with only two numeric values (for multiplication) or one string value and one integer value (for string replication). Otherwise, Python will just display an error message.

```
>>> 'Alice' * 'Bob'
Traceback (most recent call last):
  File "<pyshell#32>", line 1, in <module>
    'Alice' * 'Bob'
TypeError: can't multiply sequence by non-int of type 'str'
>>> 'Alice' * 5.0
Traceback (most recent call last):
  File "<pyshell#33>", line 1, in <module>
    'Alice' * 5.0
TypeError: can't multiply sequence by non-int of type 'float'
```

Storing Values in Variables

- A variable is like a box in the computer's memory where you can store a single value.
- If we need to use variables later, then the result must be stored in variable.

Assignment Statements

- You'll store values in variables with an assignment statement.
- An assignment statement consists of a variable name, an equal sign (called the assignment operator), and the value to be stored.
- Ex: spam = 42



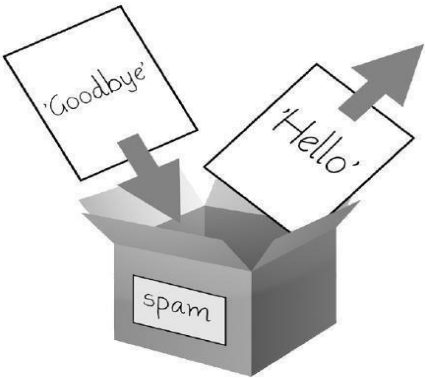
Overwriting the variable

- A variable is initialized (or created) the first time a value is stored in it ❶.
- After that, you can use it in expressions with other variables and values ❷.
- When a variable is assigned a new value ❸, the old value is forgotten, which is why spam evaluated to 42 instead of 40 at the end of the example.

```
❶ >>> spam = 40
>>> spam
40
>>> eggs = 2
❷ >>> spam + eggs
42
>>> spam + eggs + spam
82
❸ >>> spam = spam + 2
>>> spam
42
```

One more example

```
>>> spam = 'Hello'
>>> spam
'Hello'
>>> spam = 'Goodbye'
>>> spam
'Goodbye'
```



Variable names

- We can name a variable anything as long as it obeys the following three rules:
1. It can be only one word.
 2. It can use only letters, numbers, and the underscore (`_`) character.
 3. It can't begin with a number.

Table 1-3: Valid and Invalid Variable Names	
Valid variable names	Invalid variable names
balance	current-balance (hyphens are not allowed)
currentBalance	current balance (spaces are not allowed)
current_balance	4account (can't begin with a number)
_spam	42 (can't begin with a number)
SPAM	total_\$um (special characters like \$ are not allowed)
account4	'hello' (special characters like ' are not allowed)

- Variable names are case-sensitive, meaning that spam, SPAM, Spam, and sPaM are four different variables.
- This book uses camelcase for variable names instead of underscores; that is, variables look Like This instead of looking_like_this.
- A good variable name describes the data it contains.

Your First Program

- The file editor is similar to text editors such as Notepad or TextMate, but it has some specific features for typing in source code.
- The interactive shell window will always be the one with the >>> prompt.
- The file editor window will not have the >>> prompt.
- The extension for python program is .py
- Example program:

```
# This program says hello and asks for my name.

print('Hello world!')
print('What is your name?')    # ask for their name
myName = input()
print('It is good to meet you, ' + myName)
print('The length of your name is:')
print(len(myName))

print('What is your age?')    # ask for their age
myAge = input()
print('You will be ' + str(int(myAge) + 1) + ' in a year.')
```

- The output looks like:

```
Python 3.3.2 (v3.3.2:d047928ae3f6, May 16 2013, 00:06:53) [MSC v.1600 64 bit
(AMD64)] on win32
Type "copyright", "credits" or "license()" for more information.
>>> ===== RESTART =====
>>>
Hello world!
What is your name?
Al
It is good to meet you, Al
The length of your name is:
2
What is your age?
4
You will be 5 in a year.
>>>
```

Dissecting Your Program

Comments

- The following line is called a *comment*.

```
# This program says hello and asks for my name.
```

- Python ignores comments, and we can use them to write notes or remind ourselves what the code is trying to do.
- Any text for the rest of the line following a hash mark (#) is part of a comment.
- Sometimes, programmers will put a # in front of a line of code to temporarily remove it while testing a program. This is called *commenting out* code, and it can be useful when you're trying to figure out why a program doesn't work.
- Python also ignores the blank line after the comment.

The print() Function

- The print() function displays the string value inside the parentheses on the screen.

```
❷ print('Hello world!')
   print('What is your name?') # ask for their name
```

- The line print ('Hello world!') means “Print out the text in the string 'Hello world!'.”
- When Python executes this line, you say that Python is *calling* the print () function and the string value is being *passed* to the function.
- A value that is passed to a function call is an *argument*.
- The quotes are not printed to the screen. They just mark where the string begins and ends; they are not part of the string value.

Note:

We can also use this function to put a blank line on the screen; just call print () with nothing in between the parentheses.

The Input Function

- The input () function waits for the user to type some text on the keyboard and press ENTER.

```
❸ myName = input()
```

- This function call evaluates to a string equal to the user's text, and the previous line of code assigns the myName variable to this string value.
- We can think of the input() function call as an expression that evaluates to whatever string the user typed in. If the user entered 'Al', then the expression would evaluate to myName = 'Al'.

Printing the User's Name

- The following call to print() actually contains the expression 'It is good to meet you, ' + myName between the parentheses.

```
❹ print('It is good to meet you, ' + myName)
```

- Remember that expressions can always evaluate to a single value.
- If 'Al' is the value stored in myName on the previous line, then this expression evaluates to 'It is good to meet you, Al'.
- This single string value is then passed to print(), which prints it on the screen.

The len() Function

- We can pass the len() function a string value (or a variable containing a string), and the function evaluates to the integer value of the number of characters in that string.

```
❹ print('The length of your name is:')
   print(len(myName))
```

- In the interactive shell:

```
>>> len('hello')
5
>>> len('My very energetic monster just scarfed nachos.')
46
>>> len('')
0
```

- len(myName) evaluates to an integer. It is then passed to print() to be displayed on the screen.
- Possible errors: The print() function isn't causing that error, but rather it's the expression you tried to pass to print().

```
>>> print('I am ' + 29 + ' years old.')
Traceback (most recent call last):
  File "<pyshell#6>", line 1, in <module>
    print('I am ' + 29 + ' years old.')
TypeError: Can't convert 'int' object to str implicitly
```

- Python gives an error because we can use the + operator only to add two integers together or concatenate two strings. We can't add an integer to a string because this is ungrammatical in Python.

```
>>> 'I am ' + 29 + ' years old.'
Traceback (most recent call last):
  File "<pyshell#7>", line 1, in <module>
    'I am ' + 29 + ' years old.'
TypeError: Can't convert 'int' object to str implicitly
```

The str(), int() and float() Functions

- If we want to concatenate an integer such as 29 with a string to pass to print(), we'll need to get the value '29', which is the string form of 29.
- The str() function can be passed an integer value and will evaluate to a string value version of it, as follows:

```
>>> str(29)
'29'
>>> print('I am ' + str(29) + ' years old.')
I am 29 years old.
```

- Because `str(29)` evaluates to `'29'`, the expression `'I am ' + str(29) + ' years old.'` evaluates to `'I am ' + '29' + ' years old.'`, which in turn evaluates to `'I am 29 years old.'`. This is the value that is passed to the `print()` function.
- The `str()`, `int()`, and `float()` functions will evaluate to the string, integer, and floating-point forms of the value you pass, respectively.
- Converting some values in the interactive shell with these functions:

<pre>>>> str(0) '0' >>> str(-3.14) '-3.14' >>> int('42') 42 >>> int('-99') -99</pre>	<pre>>>> int(1.25) 1 >>> int(1.99) 1 >>> float('3.14') 3.14 >>> float(10) 10.0</pre>
--	--

- The previous examples call the `str()`, `int()`, and `float()` functions and pass them values of the other data types to obtain a string, integer, or floating-point form of those values.
- The `str()` function is handy when you have an integer or float that you want to concatenate to a string.
- The `int()` function is also helpful if we have a number as a string value that you want to use in some mathematics.
- For example, the `input()` function always returns a string, even if the user enters a number.
- Enter **`spam = input()`** into the interactive shell and enter **101** when it waits for your text.

```
>>> spam = input()
101
>>> spam
'101'
```

- The value stored inside `spam` isn't the integer 101 but the string `'101'`.
- If we want to do math using the value in `spam`, use the `int()` function to get the integer form of `spam` and then store this as the new value in `spam`.

```
>>> spam = int(spam)
>>> spam
101
```

- Now we should be able to treat the `spam` variable as an integer instead of a string.

```
>>> spam * 10 / 5
202.0
```

- Note that if we pass a value to `int()` that it cannot evaluate as an integer, Python will display an error message.

```
>>> int('99.99')
Traceback (most recent call last):
  File "<pyshell#18>", line 1, in <module>
    int('99.99')
ValueError: invalid literal for int() with base 10: '99.99'
>>> int('twelve')
Traceback (most recent call last):
  File "<pyshell#19>", line 1, in <module>
    int('twelve')
ValueError: invalid literal for int() with base 10: 'twelve'
```

- The int() function is also useful if we need to round a floating-point number down. If we want to round a floating-point number up, just add 1 to it afterward.

```
>>> int(7.7)
7
>>> int(7.7) + 1
8
```

- In your program, we used the int() and str() functions in the last three lines to get a value of the appropriate data type for the code.

```
⑥ print('What is your age?') # ask for their age
myAge = input()
print('You will be ' + str(int(myAge) + 1) + ' in a year.')
```

- The myAge variable contains the value returned from input().
- Because the input() function always returns a string (even if the user typed in a number), we can use the int(myAge) code to return an integer value of the string in myAge.
- This integer value is then added to 1 in the expression int(myAge) + 1.
- The result of this addition is passed to the str() function: str(int(myAge) + 1).
- The string value returned is then concatenated with the strings 'You will be ' and ' in a year.' to evaluate to one large string value.
- This large string is finally passed to print() to be displayed on the screen.

Another input:

- Let's say the user enters the string '4' for myAge.
- The string '4' is converted to an integer, so you can add one to it. The result is 5.
- The str() function converts the result back to a string, so we can concatenate it with the second string, 'in a year.', to create the final message. These evaluation steps would look something like below:

```

print('You will be ' + str(int(myAge) + 1) + ' in a year.')
print('You will be ' + str(int( '4' ) + 1) + ' in a year.')
print('You will be ' + str(    4 + 1    ) + ' in a year.')
print('You will be ' + str(        5      ) + ' in a year.')
print('You will be ' +          '5'        + ' in a year.')
print('You will be 5'                + ' in a year.')
print('You will be 5 in a year.')

```

Figure 1-4: The evaluation steps, if 4 was stored in myAge

Text and Number Equivalence

- Although the string value of a number is considered a completely different value from the integer or floating-point version, an integer can be equal to a floating point.

```

>>> 42 == '42'
False
>>> 42 == 42.0
True
>>> 42.0 == 0042.000
True

```

CHAPTER 2: FLOW CONTROL

1. Boolean Values
2. Comparison Operators
3. Boolean Operators
4. Mixing Boolean and Comparison Operators
5. Elements of Flow Control
6. Program Execution
7. Flow Control Statements
8. Importing Modules
9. Ending a Program Early with sys.exit()

Introduction

- Flow control statements can decide which Python instructions to execute under which conditions.
- These flow control statements directly correspond to the symbols in a flowchart
- In a flowchart, there is usually more than one way to go from the start to the end.
- Flowcharts represent these branching points with diamonds, while the other steps are represented with rectangles.

- The starting and ending steps are represented with rounded rectangles.

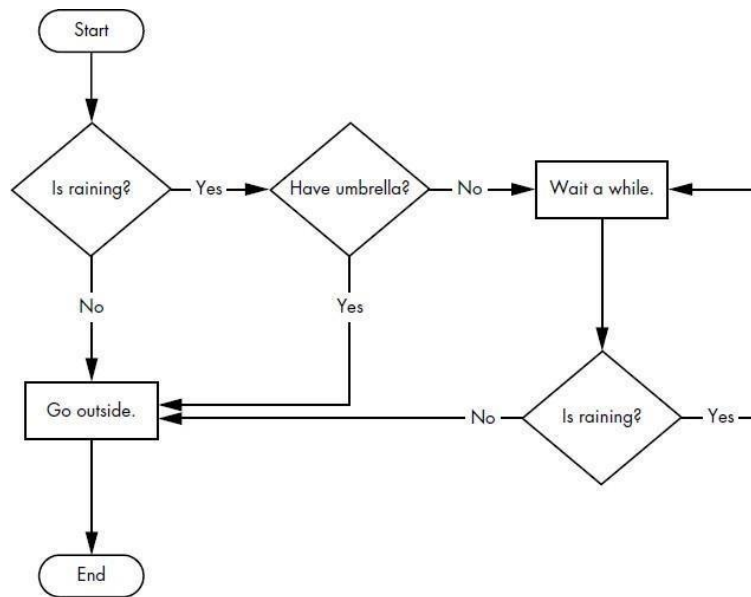


Figure 2-1: A flowchart to tell you what to do if it is raining

Boolean Values

- The Boolean data type has only two values: True and False.
- When typed as Python code, the Boolean values True and False lack the quotes you place around strings, and they always start with a capital T or F, with the rest of the word in lowercase.
- Examples:

```

❶ >>> spam = True
    >>> spam
    True
❷ >>> true
    Traceback (most recent call last):
      File "<pyshell#2>", line 1, in <module>
        true
    NameError: name 'true' is not defined
❸ >>> True = 2 + 2
    SyntaxError: assignment to keyword
  
```

- Like any other value, Boolean values are used in expressions and can be stored in variables ❶. If we don't use the proper case ❷ or we try to use True and False for variable names ❸, Python will give you an error message.

Comparison Operators

- Comparison operators compare two values and evaluate down to a single Boolean value. Table 2-1 lists the comparison operators.

Operator	Meaning
==	Equal to
!=	Not equal to
<	Less than
>	Greater than
<=	Less than or equal to
>=	Greater than or equal to

- These operators evaluate to True or False depending on the values we give them.

```
>>> 42 == 42
True
>>> 42 == 99
False
>>> 2 != 3
True
>>> 2 != 2
False
```

- The == and != operators can actually work with values of any data type.

```
>>> 'hello' == 'hello'
True
>>> 'hello' == 'Hello'
False
>>> 'dog' != 'cat'
True
>>> True == True
True
>>> True != False
True
>>> 42 == 42.0
True
❶ >>> 42 == '42'
False
```

- Note that an integer or floating-point value will always be unequal to a string value. The expression `42 == '42'` ❶ evaluates to False because Python considers the integer 42 to be different from the string '42'.
- The <, >, <=, and >= operators, on the other hand, work properly only with integer and floating-point values.

```
>>> 42 < 100
True
>>> 42 > 100
False
>>> 42 < 42
False
>>> eggCount = 42
❶ >>> eggCount <= 42
True
>>> myAge = 29
❷ >>> myAge >= 10
True
```

The Difference Between the == and = Operators

- The == operator (equal to) asks whether two values are the same as each other.
- The = operator (assignment) puts the value on the right into the variable on the left.
- We often use comparison operators to compare a variable's value to some other value, like in the `eggCount <= 42` ❶ and `myAge >= 10` ❷ examples.

Boolean Operators

- The three Boolean operators (and, or, and not) are used to compare Boolean values.

Binary Boolean Operators

- The and and or operators always take two Boolean values (or expressions), so they're considered binary Operators.

and operator: The and operator evaluates an expression to True if both Boolean values are True; otherwise, it evaluates to False.

Table 2-2: The and Operator's Truth Table

Expression	Evaluates to...
True and True	True
True and False	False
False and True	False
False and False	False

```
>>> True and True
True
>>> True and False
False
```

or operator: The or operator evaluates an expression to True if either of the two Boolean values is True. If both are False, it evaluates to False.

Table 2-3: The or Operator's Truth Table

Expression	Evaluates to...
True or True	True
True or False	True
False or True	True
False or False	False

```
>>> False or True
True
>>> False or False
False
```

not operator: The not operator operates on only one Boolean value (or expression). The not operator simply evaluates to the opposite Boolean value. Much like using double negatives in speech and writing, you can nest not operators ❶, though there's never not no reason to do this in real programs.

Table 2-4: The not Operator's Truth Table

Expression	Evaluates to...
not True	False
not False	True

```
>>> not True
False
❶ >>> not not not not True
True
```

Mixing Boolean and Comparison Operators

- Since the comparison operators evaluate to Boolean values, we can use them in expressions with the Boolean operators. Ex:

```
>>> (4 < 5) and (5 < 6)
True
>>> (4 < 5) and (9 < 6)
False
>>> (1 == 2) or (2 == 2)
True
```

- The computer will evaluate the left expression first, and then it will evaluate the right expression. When it knows the Boolean value for each, it will then evaluate the whole expression down to one Boolean value. You can think of the computer's evaluation process for (4 < 5) and (5 < 6) as shown in Figure below:

```

(4 < 5) and (5 < 6)
  ↓
True and (5 < 6)
  ↓
True and True
  ↓
True

```

- We can also use multiple Boolean operators in an expression, along with the comparison operators.

```

>>> 2 + 2 == 4 and not 2 + 2 == 5 and 2 * 2 == 2 + 2
True

```

- The Boolean operators have an order of operations just like the math operators do. After any math and comparison operators evaluate, Python evaluates the not operators first, then the and operators, and then the or operators.

Elements of Flow Control

- Flow control statements often start with a part called the condition, and all are followed by a block of code called the clause.

Conditions:

- The Boolean expressions you've seen so far could all be considered conditions, which are the same thing as expressions; condition is just a more specific name in the context of flow control statements.
- Conditions always evaluate down to a Boolean value, True or False.
- A flow control statement decides what to do based on whether its condition is True or False, and almost every flow control statement uses a condition.

Blocks of Code:

- Lines of Python code can be grouped together in blocks. There are three rules for blocks.
 1. Blocks begin when the indentation increases.
 2. Blocks can contain other blocks.
 3. Blocks end when the indentation decreases to zero or to a containing block's indentation.

```

if name == 'Mary':
❶    print('Hello Mary')
    if password == 'swordfish':
❷        print('Access granted.')
    else:
❸        print('Wrong password.')

```

- The first block of code ❶ starts at the line `print('Hello Mary')` and contains all the lines after it. Inside this block is another block ❷, which has only a single line in it: `print('Access Granted.')`. The third block ❸ is also one line long: `print('Wrong password.')`.

Program Execution:

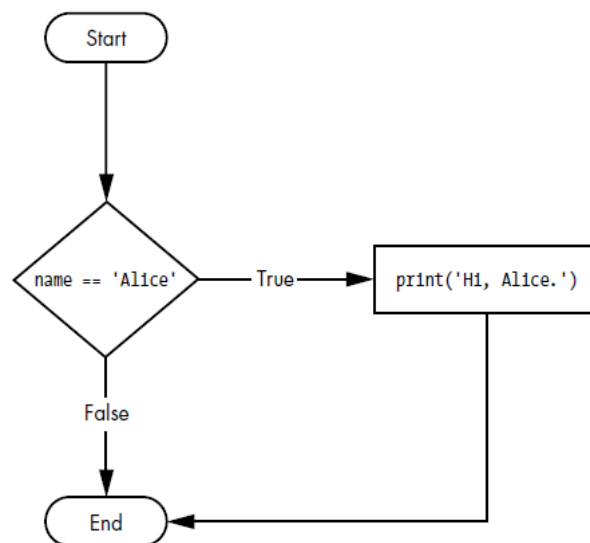
- The program execution (or simply, execution) is a term for the current instruction being executed.

Flow Control Statements:**1. if Statements:**

- The most common type of flow control statement is the if statement.
- An if statement's clause (that is, the block following the if statement) will execute if the statement's condition is True. The clause is skipped if the condition is False.
- In plain English, an if statement could be read as, "If this condition is true, execute the code in the clause." In Python, an if statement consists of the following:
 1. The if keyword
 2. A condition (that is, an expression that evaluates to True or False)
 3. A colon
 4. Starting on the next line, an indented block of code (called the if clause)
- Example:

```
if name == 'Alice':
    print('Hi, Alice.')
```

- Flowchart:

**2. else Statements:**

- An if clause can optionally be followed by an else statement. The else clause is executed only when the if statement's condition is False.
- In plain English, an else statement could be read as, "If this condition is true, execute this code. Or else, execute that code."
- An else statement doesn't have a condition, and in code, an else statement always consists of the following:
 1. The else keyword
 2. A colon
 3. Starting on the next line, an indented block of code (called the else clause)

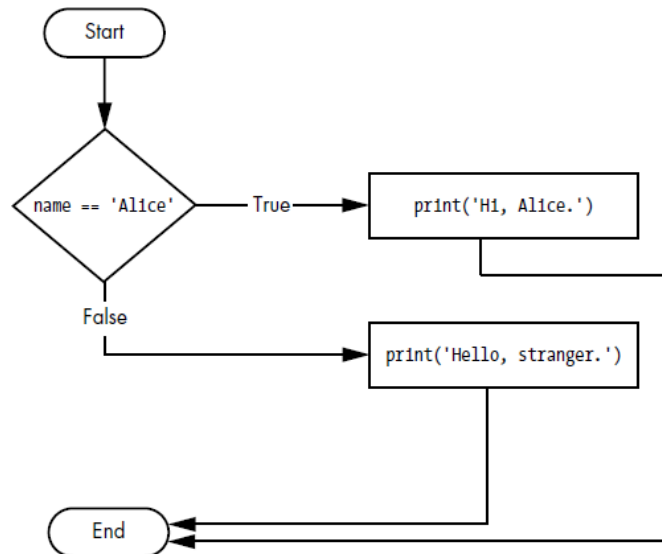
➤ Example:

```

if name == 'Alice':
    print('Hi, Alice.')
else:
    print('Hello, stranger.')

```

➤ Flowchart:

3. **elif Statements:**

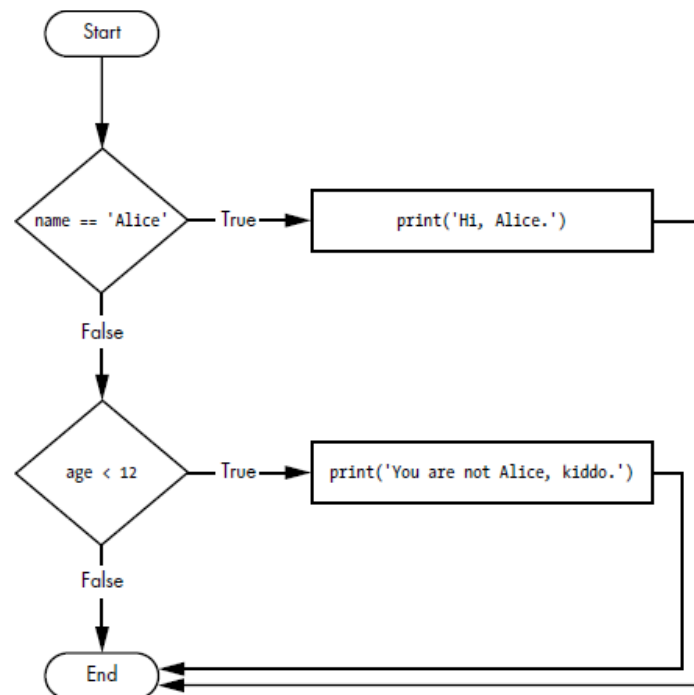
- While only one of the if or else clauses will execute, we may have a case where we want one of many possible clauses to execute.
- The elif statement is an “else if” statement that always follows an if or another elif statement.
- It provides another condition that is checked only if all of the previous conditions were False.
- In code, an elif statement always consists of the following:
 1. The elif keyword
 2. A condition (that is, an expression that evaluates to True or False)
 3. A colon
 4. Starting on the next line, an indented block of code (called the elif clause)
- Example:

```

if name == 'Alice':
    print('Hi, Alice.')
elif age < 12:
    print('You are not Alice, kiddo.')

```

➤ Flowchart:

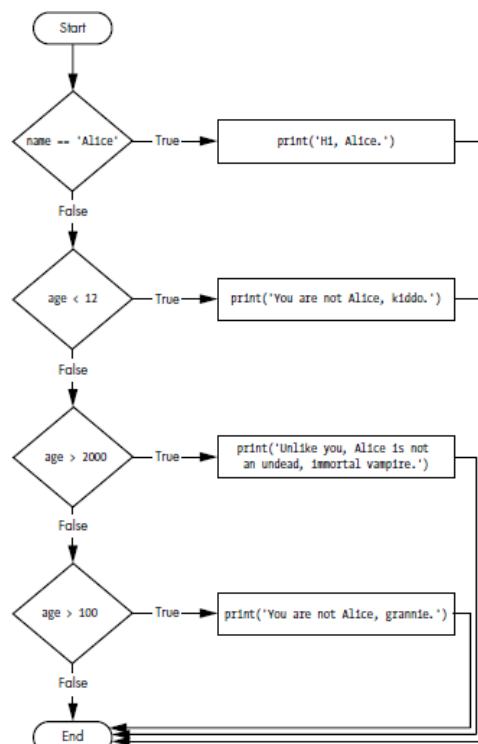


- When there is a chain of elif statements, only one or none of the clauses will be executed.
- Example:

```

if name == 'Alice':
    print('Hi, Alice.')
elif age < 12:
    print('You are not Alice, kiddo.')
elif age > 2000:
    print('Unlike you, Alice is not an undead, immortal vampire.')
elif age > 100:
    print('You are not Alice, grannie.')
  
```

- Flowchart:



- The *order of the elif statements does matter*, however. Let's see by rearranging the previous code.
- Say the age variable contains the value 3000 before this code is executed.

```

if name == 'Alice':
    print('Hi, Alice.')
elif age < 12:
    print('You are not Alice, kiddo.')
❶ elif age > 100:
    print('You are not Alice, grannie.')
elif age > 2000:
    print('Unlike you, Alice is not an undead, immortal vampire.')

```

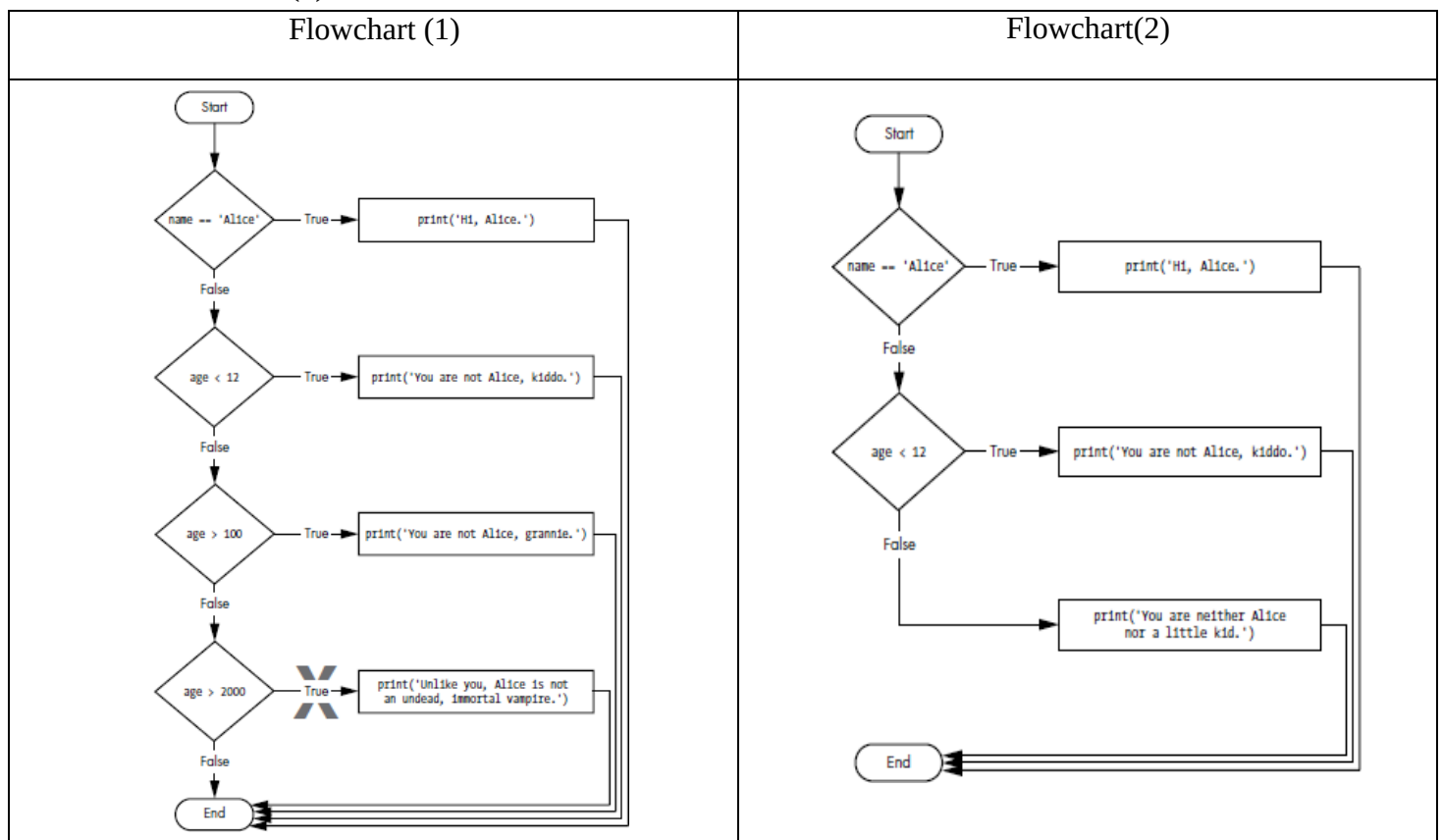
- We might expect the code to print the string 'Unlike you, Alice is not an undead, immortal vampire.'.
- However, because the age > 100 condition is True (after all, 3000 is greater than 100) ❶, the string 'You are not Alice, grannie.' is printed, and the rest of the elif statements are automatically skipped.
- Remember, at most only one of the clauses will be executed, and for elif statements, the order matters!
- Flowchart → (1)
- Optionally, we can have an else statement after the last elif statement.
- In that case, it is guaranteed that at least one (and only one) of the clauses will be executed.
- If the conditions in every if and elif statement are False, then the else clause is executed.
- In plain English, this type of flow control structure would be, "If the first condition is true, do this. Else, if the second condition is true, do that. Otherwise, do something else."
- Example:

```

if name == 'Alice':
    print('Hi, Alice.')
elif age < 12:
    print('You are not Alice, kiddo.')
else:
    print('You are neither Alice nor a little kid.')

```

- Flowchart → (2)

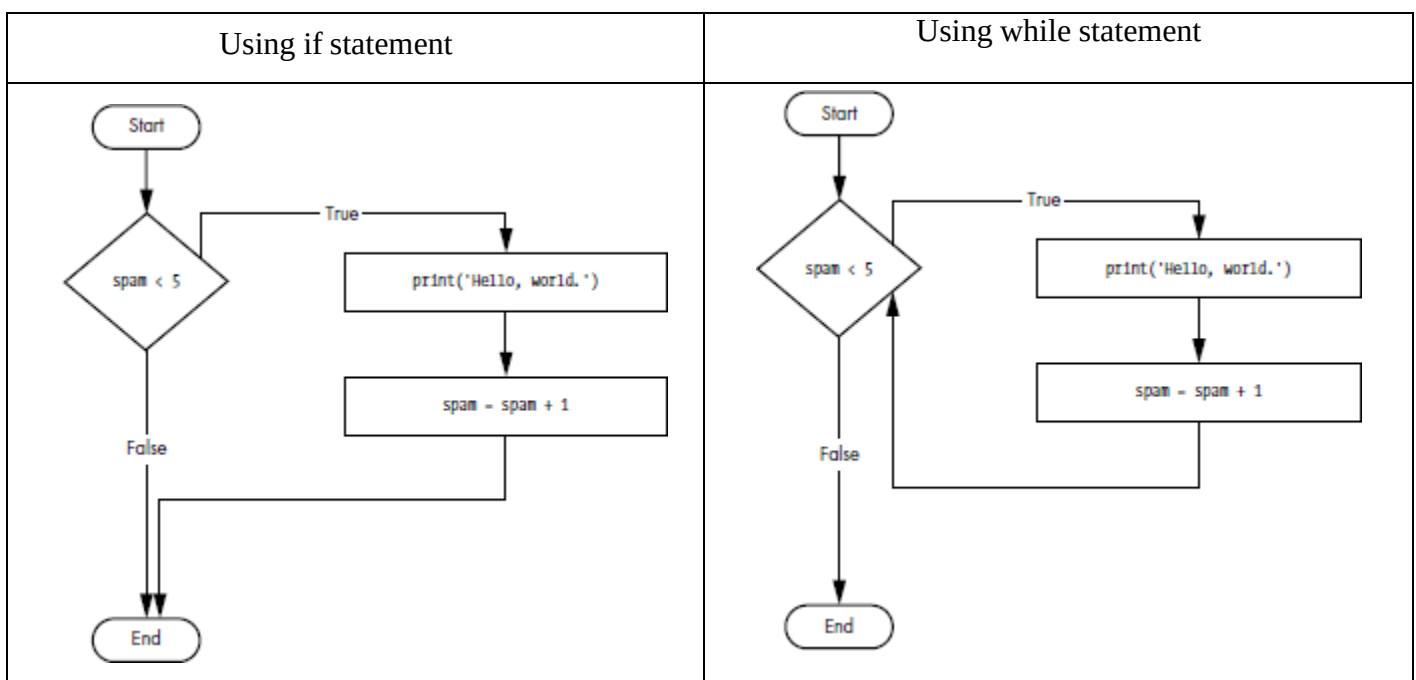


4. while loop Statements:

- We can make a block of code execute over and over again with a while statement.
- The code in a while clause will be executed as long as the while statement's condition is True.
- In code, a while statement always consists of the following:
 1. The while keyword
 2. A condition (that is, an expression that evaluates to True or False).
 3. A colon
 4. Starting on the next line, an indented block of code (called the while clause)
- We can see that a while statement looks similar to an if statement. The difference is in how they behave. At the end of an if clause, the program execution continues after the if statement.
- But, at the end of a while clause, the program execution jumps back to the start of the while statement. The while clause is often called the while loop or just the loop.
- Example:

Using if statement	Using while statement
<pre>spam = 0 if spam < 5: print('Hello, world.') spam = spam + 1</pre>	<pre>spam = 0 while spam < 5: print('Hello, world.') spam = spam + 1</pre>

- These statements are similar—both if and while check the value of spam, and if it's less than five, they print a message.
- But when we run these two code snippets, for the if statement, the output is simply "Hello, world."
- But for the while statement, it's "Hello, world." repeated five times!
- Flowchart:



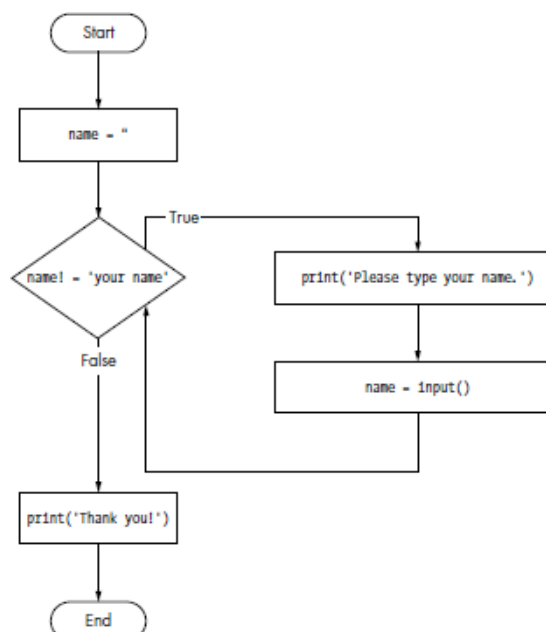
- In the while loop, the condition is always checked at the start of each iteration (that is, each time the loop is executed).
- If the condition is True, then the clause is executed, and afterward, the condition is checked again.
- The first time the condition is found to be False, the while clause is skipped.

An annoying while loop:

- Here's a small example program that will keep asking to type, literally, your name.

Example Program	Output
<pre> ❶ name = '' ❷ while name != 'your name': print('Please type your name.') ❸ name = input() ❹ print('Thank you!') </pre>	<pre> Please type your name. A1 Please type your name. Albert Please type your name. %#@#X*(^&!!! Please type your name. your name Thank you! </pre>

- First, the program sets the name variable ❶ to an empty string.
- This is so that the name != 'your name' condition will evaluate to True and the program execution will enter the while loop's clause ❷.
- The code inside this clause asks the user to type their name, which is assigned to the name variable ❸.
- Since this is the last line of the block, the execution moves back to the start of the while loop and reevaluates the condition.
- If the value in name is not equal to the string 'your name', then the condition is True, and the execution enters the while clause again.
- But once the user types your name, the condition of the while loop will be 'your name' != 'your name', which evaluates to False.
- The condition is now False, and instead of the program execution reentering the while loop's clause, it skips past it and continues running the rest of the program ❹.
- Flowchart:

**5. break Statements:**

- There is a shortcut to getting the program execution to break out of a while loop's clause early.
- If the execution reaches a break statement, it immediately exits the while loop's clause.

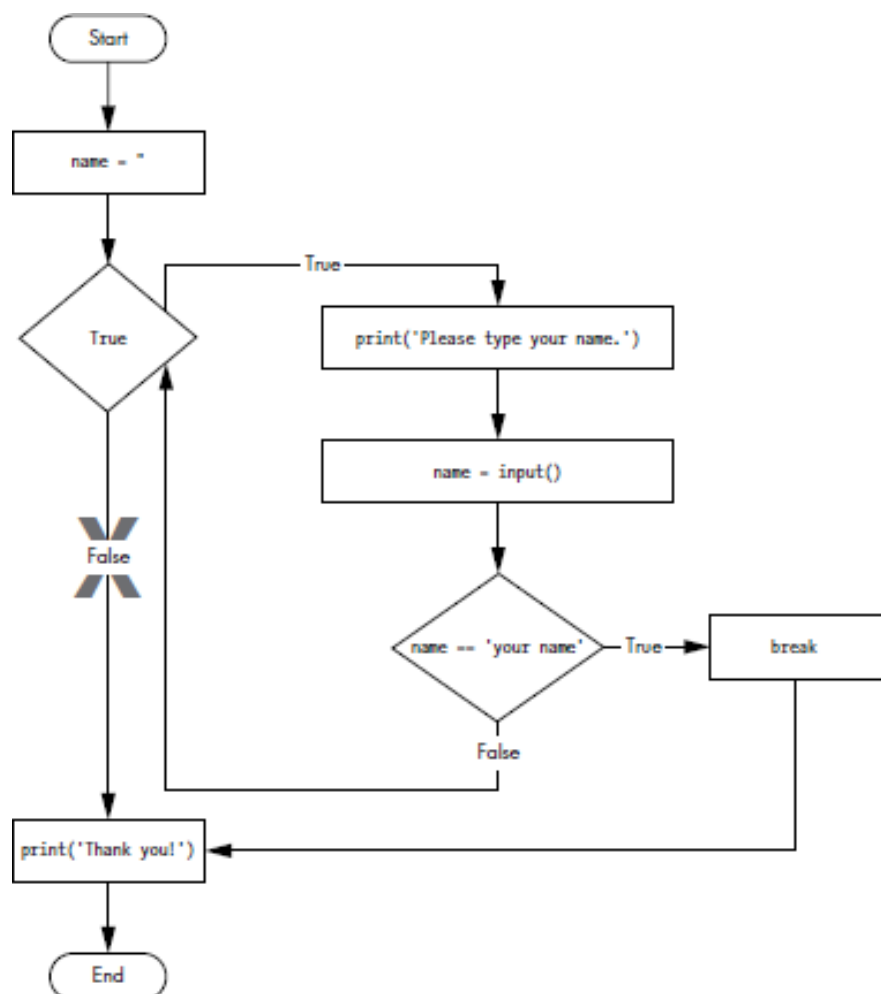
- In code, a break statement simply contains the break keyword.
- Example:

```

❶ while True:
    print('Please type your name.')
❷     name = input()
❸     if name == 'your name':
❹         break
❺ print('Thank you!')

```

- The first line ❶ creates an infinite loop; it is a while loop whose condition is always True. (The expression True, after all, always evaluates down to the value True.)
- The program execution will always enter the loop and will exit it only when a break statement is executed. (An infinite loop that never exits is a common programming bug.)
- Just like before, this program asks the user to type your name ❷.
- Now, however, while the execution is still inside the while loop, an if statement gets executed ❸ to check whether name is equal to your name.
- If this condition is True, the break statement is run ❹, and the execution moves out of the loop to print('Thank you!') ❺.
- Otherwise, the if statement's clause with the break statement is skipped, which puts the execution at the end of the while loop.
- At this point, the program execution jumps back to the start of the while statement ❶ to recheck the condition. Since this condition is merely the True Boolean value, the execution enters the loop to ask the user to type your name again.
- Flowchart:



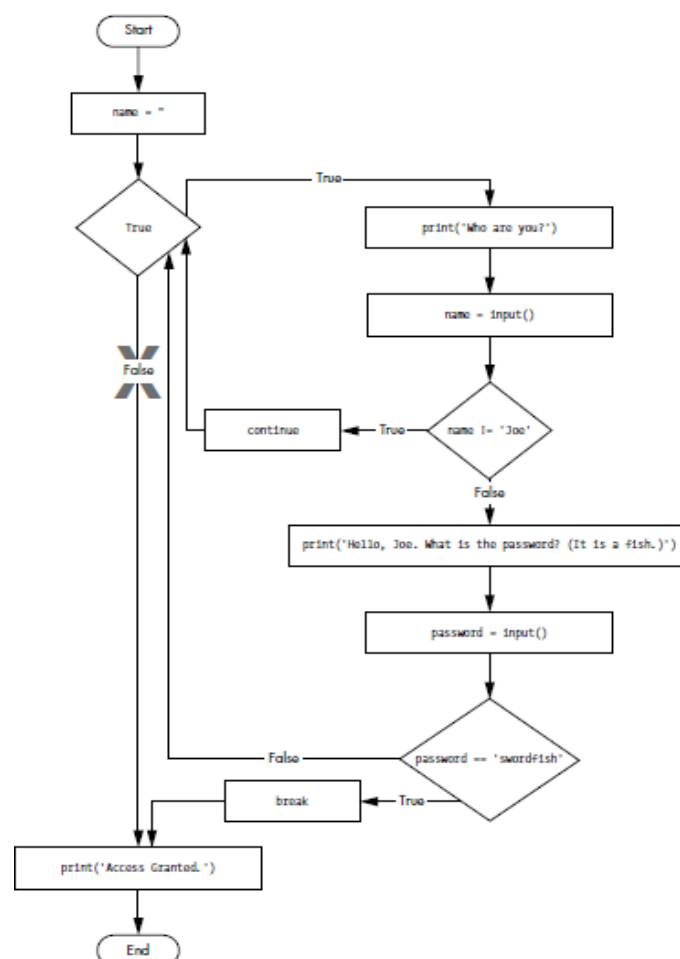
6. Continue statements:

- Like break statements, continue statements are used inside loops.
- When the program execution reaches a continue statement, the program execution immediately jumps back to the start of the loop and reevaluates the loop's condition.
- Example and Output:

```
while True:
    print('Who are you?')
    name = input()
    ❶ if name != 'Joe':
    ❷     continue
    print('Hello, Joe. What is the password? (It is a fish.)')
    ❸ password = input()
    if password == 'swordfish':
    ❹     break
    ❺ print('Access granted.')
```

```
Who are you?
I'm fine, thanks. Who are you?
Who are you?
Joe
Hello, Joe. What is the password? (It is a fish.)
Mary
Who are you?
Joe
Hello, Joe. What is the password? (It is a fish.)
swordfish
Access granted.
```

- If the user enters any name besides Joe ❶, the continue statement ❷ causes the program execution to jump back to the start of the loop.
- When it reevaluates the condition, the execution will always enter the loop, since the condition is simply the value True. Once they make it past that if statement, the user is asked for a password ❸.
- If the password entered is swordfish, then the break statement ❹ is run, and the execution jumps out of the while loop to print Access granted ❺.
- Otherwise, the execution continues to the end of the while loop, where it then jumps back to the start of the loop.
- Flowchart:



- There are some values in other data types that conditions will consider equivalent to True and False.
- When used in conditions, 0, 0.0, and "" (the empty string) are considered False, while all other values are considered True.
- Example:

```
name = ''
while not name:❶
    print('Enter your name:')
    name = input()
print('How many guests will you have?')
numOfGuests = int(input())
if numOfGuests:❷
    print('Be sure to have enough room for all your guests.')❸
print('Done')
```

NOTE: If you ever run a program that has a bug causing it to get stuck in an infinite loop, press CTRL-C. This will send a KeyboardInterrupt error to your program and cause it to stop immediately.

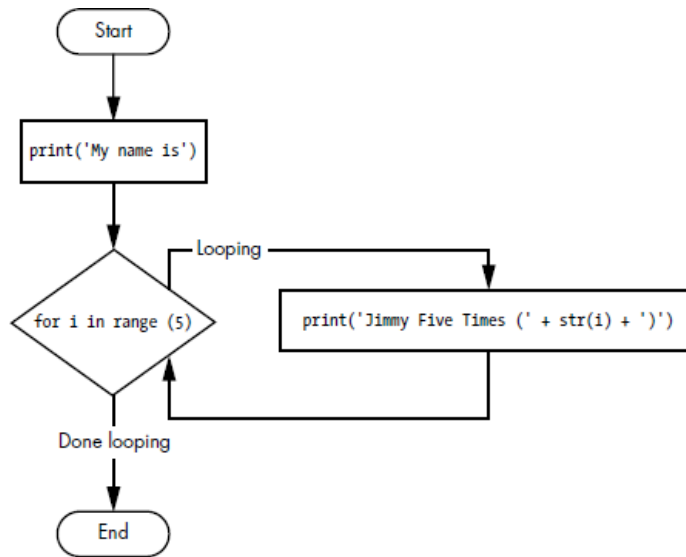
7. for loops and the range() function:

- If we want to execute a block of code only a certain number of times then we can do this with a for loop statement and the range() function.
- In code, a for statement looks something like for i in range(5): and always includes the following:
 1. The for keyword
 2. A variable name
 3. The in keyword
 4. A call to the range() method with up to three integers passed to it
 5. A colon
 6. Starting on the next line, an indented block of code (called the for clause)
- Example and output:

```
print('My name is')
for i in range(5):
    print('Jimmy Five Times (' + str(i) + ')')
```

```
My name is
Jimmy Five Times (0)
Jimmy Five Times (1)
Jimmy Five Times (2)
Jimmy Five Times (3)
Jimmy Five Times (4)
```

- The code in the for loop's clause is run five times.
- The first time it is run, the variable i is set to 0.
- The print() call in the clause will print Jimmy Five Times (0).
- After Python finishes an iteration through all the code inside the for loop's clause, the execution goes back to the top of the loop, and the for statement increments i by one.
- This is why range(5) results in five iterations through the clause, with i being set to 0, then 1, then 2, then 3, and then 4.
- The variable i will go up to, but will not include, the integer passed to range().
- Flowchart:



➤ Example 2:

```

❶ total = 0
❷ for num in range(101):
❸     total = total + num
❹ print(total)
  
```

- The result should be 5,050. When the program first starts, the total variable is set to 0 ❶.
- The for loop ❷ then executes `total = total + num` ❸ 100 times.
- By the time the loop has finished all of its 100 iterations, every integer from 0 to 100 will have been added to total. At this point, total is printed to the screen ❹.

An equivalent while loop: For the first example of for loop.

```

print('My name is')
i = 0
while i < 5:
    print('Jimmy Five Times (' + str(i) + '))'
    i = i + 1
  
```

8. **The Starting, Stopping, and Stepping Arguments to range()**

- Some functions can be called with multiple arguments separated by a comma, and `range()` is one of them.
- This lets us change the integer passed to `range()` to follow any sequence of integers, including starting at a number other than zero.

```

for i in range(12, 16):
    print(i)
  
```

- The first argument will be where the for loop's variable starts, and the second argument will be up to, but not including, the number to stop at.

```

12
13
14
15
  
```

- The range() function can also be called with three arguments. The first two arguments will be the start and stop values, and the third will be the step argument. The step is the amount that the variable is increased by after each iteration.

```
for i in range(0, 10, 2):
    print(i)
```

- So calling range(0, 10, 2) will count from zero to eight by intervals of two.

```
0
2
4
6
8
```

- The range() function is flexible in the sequence of numbers it produces for for loops. We can even use a negative number for the step argument to make the for loop count down instead of up.

```
for i in range(5, -1, -1):
    print(i)
```

- Running a for loop to print i with range(5, -1, -1) should print from five down to zero.

```
5
4
3
2
1
0
```

Importing Modules

- All Python programs can call a basic set of functions called built-in functions, including the print(), input(), and len() functions.
- Python also comes with a set of modules called the standard library.
- Each module is a Python program that contains a related group of functions that can be embedded in your programs.
- For example, the math module has mathematics-related functions, the random module has random number-related functions, and so on.
- Before we can use the functions in a module, we must import the module with an import statement. In code, an import statement consists of the following:
 1. The import keyword
 2. The name of the module
 3. Optionally, more module names, as long as they are separated by commas
- Once we import a module, we can use all the functions of that module.
- Example with output:

```
import random
for i in range(5):
    print(random.randint(1, 10))
```

```
4
1
8
4
1
```

- The `random.randint()` function call evaluates to a random integer value between the two integers that you pass it.
- Since `randint()` is in the `random` module, we must first type `random.` in front of the function name to tell Python to look for this function inside the `random` module.
- Here's an example of an import statement that imports four different modules:

```
import random, sys, os, math
```

- Now we can use any of the functions in these four modules.

from import Statements

- An alternative form of the import statement is composed of the `from` keyword, followed by the module name, the `import` keyword, and a star; for example, `from random import *`.
- With this form of import statement, calls to functions in `random` will not need the `random` prefix.
- However, using the full name makes for more readable code, so it is better to use the normal form of the import statement.

Ending a Program Early with `sys.exit()`

- The last flow control concept is how to terminate the program. This always happens if the program execution reaches the bottom of the instructions.
- However, we can cause the program to terminate, or exit, by calling the `sys.exit()` function. Since this function is in the `sys` module, we have to import `sys` before your program can use it.

```
import sys

while True:
    print('Type exit to exit.')
    response = input()
    if response == 'exit':
        sys.exit()
    print('You typed ' + response + '.')
```

- This program has an infinite loop with no `break` statement inside. The only way this program will end is if the user enters `exit`, causing `sys.exit()` to be called.
- When `response` is equal to `exit`, the program ends.
- Since the `response` variable is set by the `input()` function, the user must enter `exit` in order to stop the program.

CHAPTER 3: FUNCTIONS

1. `def` Statements with Parameters
2. Return Values and `return` Statements
3. The `None` Value
4. Keyword Arguments and `print()`

5. Local and Global Scope
6. The global Statement
7. Exception Handling
8. A Short Program: Guess the Number

Introduction

- A function is like a mini-program within a program.
- Example:

```

❶ def hello():
❷     print('Howdy!')
        print('Howdy!!!')
        print('Hello there.')

❸ hello()
    hello()
    hello()

```

- The first line is a def statement ❶, which defines a function named hello().
- The code in the block that follows the def statement ❷ is the body of the function. This code is executed when the function is called, not when the function is first defined.
- The hello() lines after the function ❸ are functioncalls.
- In code, a function call is just the function's name followed by parentheses, possibly with some number of arguments in between the parentheses.
- When the program execution reaches these calls, it will jump to the top line in the function and begin executing the code there.
- When it reaches the end of the function, the execution returns to the line that called the function and continues moving through the code as before.
- Since this program calls hello() three times, the code in the hello() function is executed three times. When we run this program, the output looks like this:

```

Howdy!
Howdy!!!
Hello there.
Howdy!
Howdy!!!
Hello there.
Howdy!
Howdy!!!
Hello there.

```

- A major purpose of functions is to group code that gets executed multiple times. Without a function defined, we would have to copy and paste this code each time, and the program would look like this:

```

print('Howdy!')
print('Howdy!!!')
print('Hello there.')
print('Howdy!')
print('Howdy!!!')
print('Hello there.')
print('Howdy!')
print('Howdy!!!')
print('Hello there.')

```

def Statements with Parameters

- When we call the print() or len() function, we pass in values, called arguments in this context, by typing them between the parentheses.
- We can also define our own functions that accept arguments.
- Example with output:

<pre> ❶ def hello(name): ❷ print('Hello ' + name) ❸ hello('Alice') hello('Bob') </pre>	<pre> Hello Alice Hello Bob </pre>
---	------------------------------------

- The definition of the hello() function in this program has a parameter called name ❶.
- A parameter is a variable that an argument is stored in when a function is called. The first time the hello() function is called, it's with the argument 'Alice' ❸.
- The program execution enters the function, and the variable name is automatically set to 'Alice', which is what gets printed by the print() statement ❷.
- One special thing to note about parameters is that the value stored in a parameter is forgotten when the function returns.

Return Values and Return Statements

- The value that a function call evaluates to is called the return value of the function.
- Ex: len('Hello') → Return values is 5
- When creating a function using the def statement, we can specify what the return value should be with a return statement.
- A return statement consists of the following:
 1. The return keyword
 2. The value or expression that the function should return.
- When an expression is used with a return statement, the return value is what this expression evaluates to.
- For example, the following program defines a function that returns a different string depending on what number it is passed as an argument.

```

❶ import random

❷ def getAnswer(answerNumber):
❸     if answerNumber == 1:
        return 'It is certain'
    elif answerNumber == 2:
        return 'It is decidedly so'
    elif answerNumber == 3:
        return 'Yes'
    elif answerNumber == 4:
        return 'Reply hazy try again'
    elif answerNumber == 5:
        return 'Ask again later'
    elif answerNumber == 6:
        return 'Concentrate and ask again'
    elif answerNumber == 7:
        return 'My reply is no'
    elif answerNumber == 8:
        return 'Outlook not so good'
    elif answerNumber == 9:
        return 'Very doubtful'

❹ r = random.randint(1, 9)
❺ fortune = getAnswer(r)
❻ print(fortune)

```

- When this program starts, Python first imports the random module ❶.
- Then the getAnswer() function is defined ❷. Because the function is being defined (and not called), the execution skips over the code in it.
- Next, the random.randint() function is called with two arguments, 1 and 9 ❸.
- It evaluates to a random integer between 1 and 9 (including 1 and 9 themselves), and this value is stored in a variable named r.
- The getAnswer() function is called with r as the argument ❹.
- The program execution moves to the top of the getAnswer() function ❺, and the value r is stored in a parameter named answerNumber.
- Then, depending on this value in answerNumber, the function returns one of many possible string values. The program execution returns to the line at the bottom of the program that originally called getAnswer() ❻.
- The returned string is assigned to a variable named fortune, which then gets passed to a print() call ❼.
- Note that since we can pass return values as an argument to another function call, we could shorten these three lines into single line as follows:

```
r = random.randint(1, 9)
fortune = getAnswer(r)
print(fortune)
```

=

```
print(getAnswer(random.randint(1, 9)))
```

The None Value

- In Python there is a value called None, which represents the absence of a value.
- None is the only value of the NoneType data type.
- This value-without-a-value can be helpful when we need to store something that won't be confused for a real value in a variable.
- One place where None is used is as the return value of print().
- The print() function displays text on the screen, but it doesn't need to return anything in the same way len() or input() does. But since all function calls need to evaluate to a return value, print() returns None.

```
>>> spam = print('Hello!')
Hello!
>>> None == spam
True
```

- Behind the scenes, Python adds return None to the end of any function definition with no return statement.

Keyword Arguments and print()

- Most arguments are identified by their position in the function call.
- For example, random.randint(1, 10) is different from random.randint(10, 1).
- The function call random.randint(1, 10) will return a random integer between 1 and 10, because the first argument is the low end of the range and the second argument is the high end while random.randint(10, 1) causes an error.
- However, keyword arguments are identified by the keyword put before them in the function call.
- Keyword arguments are often used for optional parameters.

- For example, the `print()` function has the optional parameters `end` and `sep` to specify what should be printed at the end of its arguments and between its arguments (separating them), respectively.

<pre>print('Hello') print('World')</pre>	<pre>Hello World</pre>
--	------------------------

- The two strings appear on separate lines because the `print()` function automatically adds a newline character to the end of the string it is passed.
- However, we can set the `end` keyword argument to change this to a different string.
- For example, if the program were this:

<pre>print('Hello', end='') print('World')</pre>	<pre>HelloWorld</pre>
--	-----------------------

- The output is printed on a single line because there is no longer a new-line printed after 'Hello'. Instead, the blank string is printed. This is useful if we need to disable the newline that gets added to the end of every `print()` function call.
- Similarly, when we pass multiple string values to `print()`, the function will automatically separate them with a single space.

<pre>>>> print('cats', 'dogs', 'mice') cats dogs mice</pre>
--

- But we could replace the default separating string by passing the `sep` keyword argument.

<pre>>>> print('cats', 'dogs', 'mice', sep=',') cats,dogs,mice</pre>

Local and Global Scope

- Parameters and variables that are assigned in a called function are said to exist in that function's local scope.
- Variables that are assigned outside all functions are said to exist in the global scope.
- A variable that exists in a local scope is called a local variable, while a variable that exists in the global scope is called a global variable.
- A variable must be one or the other; it cannot be both local and global.
- When a scope is destroyed, all the values stored in the scope's variables are forgotten.
- There is only one global scope, and it is created when your program begins. When your program terminates, the global scope is destroyed, and all its variables are forgotten.
- A local scope is created whenever a function is called. Any variables assigned in this function exist within the local scope. When the function returns, the local scope is destroyed, and these variables are forgotten.
- Scopes matter for several reasons:
 1. Code in the global scope cannot use any local variables.
 2. However, a local scope can access global variables.
 3. Code in a function's local scope cannot use variables in any other local scope.
 4. We can use the same name for different variables if they are in different scopes. That is, there can be a local variable named `spam` and a global variable also named `spam`.

Local Variables Cannot Be Used in the Global Scope

- Consider this program, which will cause an error when you run it:

Program

Output → Error

```
def spam():
    eggs = 31337
spam()
print(eggs)
```

```
Traceback (most recent call last):
  File "C:/test3784.py", line 4, in <module>
    print(eggs)
NameError: name 'eggs' is not defined
```

- The error happens because the eggs variable exists only in the local scope created when spam() is called.
- Once the program execution returns from spam, that local scope is destroyed, and there is no longer a variable named eggs.

Local Scopes Cannot Use Variables in Other Local Scopes

- A new local scope is created whenever a function is called, including when a function is called from another function. Consider this program:

```
def spam():
    ❶ eggs = 99
    ❷ bacon()
    ❸ print(eggs)

    def bacon():
        ham = 101
        ❹ eggs = 0

    ❺ spam()
```

- When the program starts, the spam() function is called ❺, and a local scope is created.
- The local variable eggs ❶ is set to 99.
- Then the bacon() function is called ❷, and a second local scope is created. Multiple local scopes can exist at the same time.
- In this new local scope, the local variable ham is set to 101, and a local variable eggs—which is different from the one in spam()'s local scope—is also created ❹ and set to 0.
- When bacon() returns, the local scope for that call is destroyed. The program execution continues in the spam() function to print the value of eggs ❸, and since the local scope for the call to spam() still exists here, the eggs variable is set to 99.

Global Variables Can Be Read from a Local Scope

- Consider the following program:

```
def spam():
    print(eggs)
eggs = 42
spam()
print(eggs)
```

- Since there is no parameter named eggs or any code that assigns eggs a value in the spam() function, when eggs is used in spam(), Python considers it a reference to the global variable eggs. This is why 42 is printed when the previous program is run.

Local and Global Variables with the Same Name

- To simplify, avoid using local variables that have the same name as a global variable or another local variable.
- But technically, it's perfectly legal to do so.

Example

Output

```
def spam():
    ❶ eggs = 'spam local'
    print(eggs)    # prints 'spam local'

def bacon():
    ❷ eggs = 'bacon local'
    print(eggs)    # prints 'bacon local'
    spam()
    print(eggs)    # prints 'bacon local'

❸ eggs = 'global'
bacon()
print(eggs)        # prints 'global'
```

```
bacon local
spam local
bacon local
global
```

- There are actually three different variables in this program, but confusingly they are all named eggs. The variables are as follows:
 - ❶ A variable named eggs that exists in a local scope when spam() is called.
 - ❷ A variable named eggs that exists in a local scope when bacon() is called.
 - ❸ A variable named eggs that exists in the global scope.
- Since these three separate variables all have the same name, it can be confusing to keep track of which one is being used at any given time. This is why we should avoid using the same variable name in different scopes.

The Global Statement

- If we need to modify a global variable from within a function, use the global statement.
- If we have a line such as global eggs at the top of a function, it tells Python, “In this function, eggs refers to the global variable, so don’t create a local variable with this name.”
- For example:

Program

Output

```
def spam():
    ❶ global eggs
    ❷ eggs = 'spam'

eggs = 'global'
spam()
print(eggs)
```

```
spam
```


- Because eggs is declared global at the top of spam() ❶, when eggs is set to 'spam' ❷, this assignment is done to the globally scoped eggs. No local eggs variable is created.
- There are four rules to tell whether a variable is in a local scope or global scope:
 1. If a variable is being used in the global scope (that is, outside of all functions), then it is always a global variable.
 2. If there is a global statement for that variable in a function, it is a global variable.
 3. Otherwise, if the variable is used in an assignment statement in the function, it is a local variable.
 4. But if the variable is not used in an assignment statement, it is a global variable.
- Example:

Program**Output**

```

def spam():
❶  global eggs
    eggs = 'spam' # this is the global

def bacon():
❷  eggs = 'bacon' # this is a local

def ham():
❸  print(eggs) # this is the global

eggs = 42 # this is the global
spam()
print(eggs)

```

```
spam
```

- In the spam() function, eggs is the global eggs variable, because there's a global statement for eggs at the beginning of the function ❶.
- In bacon(), eggs is a local variable, because there's an assignment statement for it in that function ❷.
- In ham() ❸, eggs is the global variable, because there is no assignment statement or global statement for it in that function
- In a function, a variable will either always be global or always be local. There's no way that the code in a function can use a local variable named eggs and then later in that same function use the global eggs variable.

Note

- If we ever want to modify the value stored in a global variable from in a function, we must use a global statement on that variable.
- If we try to use a local variable in a function before we assign a value to it, as in the following program, Python will give you an error.

Program**Output**

```

def spam():
❶  print(eggs) # ERROR!
    eggs = 'spam local'

❷  eggs = 'global'
    spam()

```

```

Traceback (most recent call last):
  File "C:/test3784.py", line 6, in <module>
    spam()
  File "C:/test3784.py", line 2, in spam
    print(eggs) # ERROR!
UnboundLocalError: local variable 'eggs' referenced before assignment

```

- This error happens because Python sees that there is an assignment statement for eggs in the spam() function ❶ and therefore considers eggs to be local.
- But because print(eggs) is executed before eggs is assigned anything, the local variable eggs doesn't exist. Python will not fall back to using the global eggs variable ❷.

Exception Handling

- If we don't want to crash the program due to errors instead we want the program to detect errors, handle them, and then continue to run.
- For example,

<u>Program</u>	<u>Output</u>
<pre>def spam(divideBy): return 42 / divideBy print(spam(2)) print(spam(12)) print(spam(0)) print(spam(1))</pre>	<pre>21.0 3.5 Traceback (most recent call last): File "C:/zeroDivide.py", line 6, in <module> print(spam(0)) File "C:/zeroDivide.py", line 2, in spam return 42 / divideBy ZeroDivisionError: division by zero</pre>

- A ZeroDivisionError happens whenever we try to divide a number by zero. From the line number given in the error message, we know that the return statement in spam() is causing an error.
- Errors can be handled with try and except statements.
- The code that could potentially have an error is put in a try clause. The program execution moves to the start of a following except clause if an error happens.
- We can put the previous divide-by-zero code in a try clause and have an except clause contain code to handle what happens when this error occurs.

<u>Program</u>	<u>Output</u>
<pre>def spam(divideBy): try: return 42 / divideBy except ZeroDivisionError: print('Error: Invalid argument.') print(spam(2)) print(spam(12)) print(spam(0)) print(spam(1))</pre>	<pre>21.0 3.5 Error: Invalid argument. None 42.0</pre>

- Note that any errors that occur in function calls in a try block will also be caught. Consider the following program, which instead has the spam() calls in the try block:

<u>Program</u>	<u>Output</u>
<pre>def spam(divideBy): return 42 / divideBy try: print(spam(2)) print(spam(12)) print(spam(0)) print(spam(1)) except ZeroDivisionError: print('Error: Invalid argument.')</pre>	<pre>21.0 3.5 Error: Invalid argument.</pre>

- The reason print(spam(1)) is never executed is because once the execution jumps to the code in the except clause, it does not return to the try clause. Instead, it just continues moving down as normal.

A Short program: Guess the Number

- This is a simple “guess the number” game. When we run this program, the output will look something like this:

```
I am thinking of a number between 1 and 20.
Take a guess.
10
Your guess is too low.
Take a guess.
15
Your guess is too low.
Take a guess.
17
Your guess is too high.
Take a guess.
16
Good job! You guessed my number in 4 guesses!
```

- Code for the above program is:

```
# This is a guess the number game.
import random
secretNumber = random.randint(1, 20)
print('I am thinking of a number between 1 and 20.')

# Ask the player to guess 6 times.
for guessesTaken in range(1, 7):
    print('Take a guess.')
    guess = int(input())

    if guess < secretNumber:
        print('Your guess is too low.')
    elif guess > secretNumber:
        print('Your guess is too high.')
    else:
        break    # This condition is the correct guess!

if guess == secretNumber:
    print('Good job! You guessed my number in ' + str(guessesTaken) + ' guesses!')
else:
    print('Nope. The number I was thinking of was ' + str(secretNumber))
```

- Let's look at this code line by line, starting at the top.

```
# This is a guess the number game.
import random
secretNumber = random.randint(1, 20)
```

- First, a comment at the top of the code explains what the program does.
- Then, the program imports the random module so that it can use the random.randint() function to generate a number for the user to guess.
- The return value, a random integer between 1 and 20, is stored in the variable secretNumber.

```
print('I am thinking of a number between 1 and 20.')

# Ask the player to guess 6 times.
for guessesTaken in range(1, 7):
    print('Take a guess.')
    guess = int(input())
```

- The program tells the player that it has come up with a secret number and will give the player six chances to guess it.
- The code that lets the player enter a guess and checks that guess is in a for loop that will loop at most six times.
- The first thing that happens in the loop is that the player types in a guess.
- Since `input()` returns a string, its return value is passed straight into `int()`, which translates the string into an integer value. This gets stored in a variable named `guess`.

```
if guess < secretNumber:
    print('Your guess is too low.')
elif guess > secretNumber:
    print('Your guess is too high.')
```

- These few lines of code check to see whether the guess is less than or greater than the secret number. In either case, a hint is printed to the screen.

```
else:
    break    # This condition is the correct guess!
```

- If the guess is neither higher nor lower than the secret number, then it must be equal to the secret number, in which case you want the program execution to break out of the for loop.

```
if guess == secretNumber:
    print('Good job! You guessed my number in ' + str(guessesTaken) + ' guesses!')
else:
    print('Nope. The number I was thinking of was ' + str(secretNumber))
```

- After the for loop, the previous if...else statement checks whether the player has correctly guessed the number and prints an appropriate message to the screen.
- In both cases, the program displays a variable that contains an integer value (`guessesTaken` and `secretNumber`).
- Since it must concatenate these integer values to strings, it passes these variables to the `str()` function, which returns the string value form of these integers.
- Now these strings can be concatenated with the `+` operators before finally being passed to the `print()` function call.